https://www.hslpicker.com/#ff1a1a

https://www.rapidtables.com/convert/color/hsl-to-rgb.html

Пощупаю значения сферических гармоник : $(-\infty; +\infty)$

min_value -> -1 (blue) Hue = 240 Saturation = 1 Lightness - изменяем от 50 до 100

max_value -> 1 (red) Hue = 360 Saturation = 1 Lightness - изменяем от 100 до 50

0 ~ white

Сферическая гармоника состоит из $const \cdot \underbrace{\sin(\dots) \cdot \cos(\dots)}_{\in[-1;1]} = value$

=> min_value = -const

max_value = const

if value < 0

$-const \to 0{,}5 \quad 0 \to 1 \quad \Rightarrow \quad min_value \to 0{,}5 \quad \Rightarrow \quad 0{,}5 = min_value \cdot \alpha + \beta$

$1 = 0 \cdot \alpha + \beta$

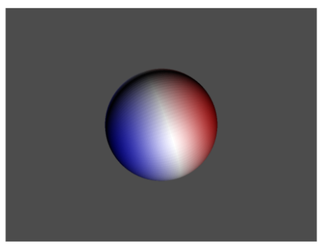
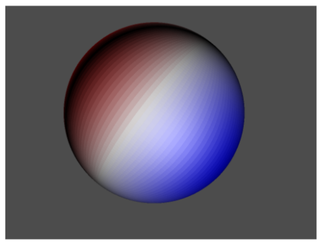
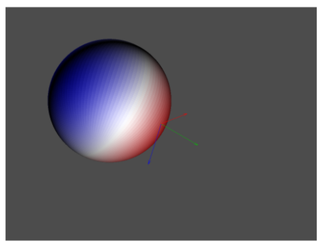
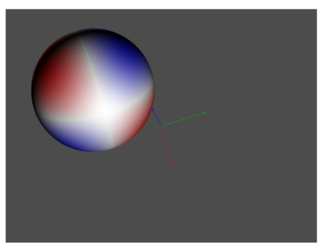
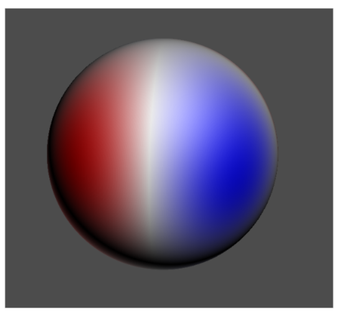
$\Rightarrow \beta = 1 \Rightarrow \alpha = \frac{0{,}5 - \beta}{min_val} = \frac{-0{,}5}{min_value} = \frac{0{,}5}{const}$

$\Rightarrow Lightness = value \cdot \frac{0{,}5}{const} + 1$

else

$0 \to 1$, $const \to 0{,}5$ $\Rightarrow$ $0 = 1 \cdot \alpha + \beta \Rightarrow \beta = 1$

$0{,}5 = max_val \cdot \alpha + \beta \Rightarrow \alpha = \frac{0{,}5 - 1}{max_val} = \frac{-0{,}5}{max_val}$

$Lightness = \frac{-0{,}5}{max_val} \cdot val + 1$

```cpp
#define _USE_MATH_DEFINES
#include <cmath>

#include <string>
#include <fstream>
#include <iostream>
#include <vector>

void dump3(std::ofstream& file, double x, double y, double z) {
	file << "(" << x << ", " << y << ", " << z << ")";
}

void dump_triangles(std::ofstream& file,
	double x1, double y1, double z1,
	double x2, double y2, double z2,
	double x3, double y3, double z3,
	float r, float g, float b) {
	dump3(file, x1, y1, z1);
	dump3(file, x2, y2, z2);
	dump3(file, x3, y3, z3);
	dump3(file, r, g, b);
	file << "\n";
}

int main() {

	std::cout << "Hello, Spherical Harmonics!" << std::endl;

	int count = 100;

	double min_theta = 0.0;
	double max_theta = M_PI;

	double min_phi = 0.0;
	double max_phi = 2.0 * M_PI;

	std::vector<double> theta_values(count);
	std::vector<double> phi_values(count);

	for (int i = 0; i < count; ++i) {

		double theta = min_theta + (max_theta - min_theta) * i / (count - 1);
		double phi = min_phi + (max_phi - min_phi) * i / (count - 1);

		theta_values[i] = theta;
		phi_values[i] = phi;
	}

	double saturation = 1.0;
	double blue_hue = 240;
	double red_hue = 360;

	std::ofstream outfile("spherical_harmonics_output.txt");
	outfile << "triangles: triangles\n";

	for (int i = 0; i < count - 1; ++i) {
		for (int j = 0; j < count - 1; ++j) {

			double x1 = std::sin(theta_values[i]) * std::cos(phi_values[j]);
			double y1 = std::sin(theta_values[i]) * std::sin(phi_values[j]);
			double z1 = std::cos(theta_values[i]);

			double x2 = std::sin(theta_values[i + 1]) * std::cos(phi_values[j]);
			double y2 = std::sin(theta_values[i + 1]) * std::sin(phi_values[j]);
			double z2 = std::cos(theta_values[i + 1]);

			double x3 = std::sin(theta_values[i]) * std::cos(phi_values[j + 1]);
			double y3 = std::sin(theta_values[i]) * std::sin(phi_values[j + 1]);
			double z3 = std::cos(theta_values[i]);

			double x4 = std::sin(theta_values[i + 1]) * std::cos(phi_values[j + 1]);
			double y4 = std::sin(theta_values[i + 1]) * std::sin(phi_values[j + 1]);
			double z4 = std::cos(theta_values[i + 1]);

			double average_theta_values = (theta_values[i] + theta_values[i + 1]) / 2.0;
			double average_phi_values = (phi_values[j] + phi_values[j + 1]) / 2.0;

			double const_value = std::sqrt(3.0 / (4.0f * M_PI));

			double value = const_value * std::cos(average_theta_values);

			double min_value = -std::abs(const_value);
			double max_value = std::abs(const_value);

			double hue = 0.0;
			double lightness = 0.0;

			float r = 0.0f;
			float g = 0.0f;
			float b = 0.0f;

			if (value < 0)
			{
				hue = blue_hue;
				lightness = 1.0 - 0.5 * (value / min_value);

				double C = (1 - std::abs(2 * lightness - 1)) * saturation;
				double X = C * (1 - std::abs(fmod(hue / 60.0, 2) - 1));
				double m = lightness - C / 2;

				r = X + m;
				g = 0 + m;
				b = C + m;
			}
			else
			{
				hue = red_hue;
				lightness = -0.5 / max_value * value + 1.0;

				double C = (1 - std::abs(2 * lightness - 1)) * saturation;
				double X = C * (1 - std::abs(fmod(hue / 60.0, 2) - 1));
				double m = lightness - C / 2;

				r = C + m;
				g = X + m;
				b = 0 + m;
			}

			// std::cout << r << ", " << g << ", " << b << std::endl;

			dump_triangles(outfile, x1, y1, z1, x2, y2, z2, x3, y3, z3, r, g, b);
			dump_triangles(outfile, x3, y3, z3, x4, y4, z4, x2, y2, z2, r, g, b);

		}
	}

	outfile.close();

	return 0;
}
```